FIRMWARE INTERN ASSIGNMENT:

PHASE 1 REPORT

Subject: Phase 1 Foundational Learning – STM32 Basics
Target Hardware: STM32F103C8T6 (Blue Pill)

Introduction

Phase 1 of this assignment was focused on developing a strong foundational understanding of embedded systems by studying the STM32F103C8T6 microcontroller at a low level. Rather than relying on high-level abstraction libraries, the emphasis was placed on understanding the internal hardware architecture of the microcontroller, including clock generation, GPIO behavior, peripheral configuration, and interrupt handling.

This phase was approached purely as a learning stage. The objective was to understand how peripherals are configured using registers, how different hardware blocks depend on each other, and how the system behaves during initial board bring-up. Reference manuals and datasheets were studied extensively to understand these interactions. This foundational understanding is essential before moving on to hands-on firmware development and debugging in later phases.

Topic 1 – GPIO Configuration & Alternate Function Mapping

GPIO configuration forms the base of almost every peripheral operation in the STM32F103. In this architecture, GPIO pins do not have predefined functionality and must be explicitly configured before use. Each pin can be configured as Input, Output, Alternate Function, or Analog mode depending on the requirement.

Understanding alternate function mapping was a key learning point. Peripherals such as USART, SPI, I2C, and timers generate signals internally, but these signals reach the outside world only when the corresponding GPIO pins are configured in Alternate Function mode. Without correct GPIO configuration, even a correctly configured peripheral remains ineffective.

From a configuration perspective, GPIO setup follows a clear flow: enabling the GPIO port clock, selecting the pin mode and configuration, and then allowing the peripheral to control the pin if alternate function mode is required. This topic reinforced that GPIO configuration is a prerequisite for peripheral operation, not an independent step.

Registers

RCC_APB2ENR enables the clock for GPIO ports.
GPIOx_CRL configures pins 0–7 for mode and function selection.
GPIOx_CRH configures pins 8–15 in the same manner.
GPIOx_BSRR provides atomic pin set and reset capability.

Topic 2 – RCC & Peripheral Clock Trees

The Reset and Clock Control (RCC) system defines how the entire microcontroller operates. Every peripheral in the STM32F103 requires a clock signal, and without explicitly enabling the clock, the peripheral remains inactive regardless of configuration.

This topic covered the internal High-Speed Internal (HSI) oscillator, the external High-Speed External (HSE) crystal, and the internal Phase Locked Loop (PLL). Understanding how the system clock is generated and distributed across the AHB, APB1, and APB2 buses clarified why different peripherals use different clock enable registers.

From a configuration standpoint, clock setup involves selecting the clock source, configuring prescalers, enabling the PLL if required, and enabling clocks for individual peripherals. This topic established that clock configuration is a system-wide dependency that directly affects timing, communication accuracy, and peripheral behavior.

Registers
RCC_CR controls HSI, HSE, and PLL enable and status.
RCC_CFGR selects the system clock source and bus prescalers.
RCC_APB1ENR enables clocks for APB1 peripherals.
RCC_APB2ENR enables clocks for APB2 peripherals.

Topic 3 – USART / UART Communication

USART peripherals in the STM32F103 were studied primarily in UART (asynchronous) mode. UART communication does not use a shared clock line, which makes it highly dependent on accurate internal clock configuration.

This topic focused on understanding how baud rate generation works, how transmit and receive functionality is controlled, and how status flags indicate communication state. The difference between polling-based communication and interrupt-driven communication was also studied conceptually.

From a configuration flow perspective, UART setup requires enabling the USART clock, configuring GPIO pins in alternate function mode, setting the baud rate based on the peripheral clock, enabling transmit and receive blocks, and finally enabling the USART peripheral itself.

Registers
USART_BRR defines the baud rate using the peripheral clock.
USART_CR1 enables the transmitter, receiver, and USART block.
USART_SR provides transmission and reception status flags.
USART_DR holds transmitted or received data.

Topic 4 – DMA (Direct Memory Access)

DMA was studied as a mechanism to transfer data between peripherals and memory without continuous CPU involvement. This allows the processor to perform other tasks while data movement happens in the background.

The distinction between Normal mode and Circular mode was an important learning point. Normal mode performs a fixed number of transfers, while Circular mode automatically restarts, making it suitable for continuous data streams.

From a configuration perspective, DMA setup requires selecting the peripheral and memory addresses, defining the number of data items to transfer, choosing the direction and data size, selecting the operating mode, and enabling the DMA channel. This topic emphasized the importance of precise configuration, as incomplete setup can prevent DMA operation without visible errors.

Registers
DMA_CPARx stores the peripheral address.
DMA_CMARx stores the memory buffer address.
DMA_CNDTRx defines the transfer count.
DMA_CCRx configures direction, mode, and channel enable.

Topic 5 – Timers (Delay, PWM, Input Capture)

Timers provide hardware-based time measurement and waveform generation. The STM32F103 includes several general-purpose timers such as TIM2, TIM3, and TIM4.

This topic covered how prescalers divide the input clock, how auto-reload values define timing periods, and how capture/compare registers control PWM duty cycle or capture external signal timing. It also reinforced the dependency between timers and GPIO alternate function configuration.

From a configuration standpoint, timer setup involves enabling the timer clock, configuring the prescaler and auto-reload values, configuring the operating mode, enabling the relevant channels, and starting the timer.

Registers
TIMx_PSC divides the timer input clock.
TIMx_ARR defines the timer period.
TIMx_CCRx controls PWM duty cycle or input capture value.
TIMx_CR1 enables and controls the timer counter.

Topic 6 – SPI Configuration & Use

SPI was studied as a synchronous, full-duplex communication protocol. The STM32F103 provides two SPI peripherals, and this topic focused on understanding master-mode operation.

Key concepts included clock polarity, clock phase, baud rate prescaling, and full-duplex data transfer. SPI configuration follows a defined flow: enabling the SPI clock, configuring GPIO pins in alternate function mode, selecting master mode, configuring timing parameters, and enabling the SPI peripheral.

This topic reinforced the importance of timing alignment between communicating devices.

Registers
SPI_CR1 configures master mode, baud rate, and clock behavior.
SPI_SR provides transmit and receive status information.
SPI_DR is used for data transmission and reception.

Topic 7 – I2C Configuration & Use

I2C communication was studied as a two-wire, address-based protocol that supports multiple devices on the same bus. The open-drain nature of the SDA and SCL lines and the requirement for pull-up resistors were key learning points.

From a configuration perspective, I2C setup involves enabling the I2C clock, configuring GPIO pins as open-drain alternate function outputs, setting the peripheral clock frequency, configuring the clock control registers, and managing Start, Address, Data, and Stop conditions.

This topic emphasized the interaction between firmware configuration and physical electrical requirements.

Registers
I2C_CR1 enables the peripheral and controls Start/Stop generation.
I2C_CR2 configures the peripheral input clock frequency.
I2C_CCR defines the I2C clock speed.
I2C_SR1 and **I2C_SR2** provide communication status information.

Topic 8 – ADC Configuration, Resolution & Sampling

The STM32F103 includes a 12-bit Analog-to-Digital Converter that converts analog voltages into digital values ranging from 0 to 4095.

This topic focused on understanding ADC resolution, sampling time, and the importance of configuring GPIO pins in Analog mode. Sampling time selection was studied to understand how signal source impedance affects conversion accuracy.

ADC configuration follows a flow that includes enabling the ADC clock, configuring input pins, selecting sampling time, enabling the ADC, and starting conversions.

Registers
ADC_CR2 enables the ADC and starts conversions.
ADC_SMPR1 and **ADC_SMPR2** configure sampling time.
ADC_DR holds the conversion result.

Topic 9 – NVIC Interrupt Structure

The Nested Vectored Interrupt Controller (NVIC) manages interrupt prioritization and execution. This topic focused on understanding how interrupt sources are mapped through the vector table and how priorities affect execution order.

Interrupt configuration conceptually involves enabling the interrupt source at the peripheral level, enabling it in the NVIC, assigning priority, and allowing global interrupt handling.

This topic helped clarify how asynchronous events are handled deterministically in STM32 systems.

Registers
NVIC_ISER enables interrupts at the core level.
NVIC_ICER disables interrupts.
NVIC_IPR assigns interrupt priority levels.

Hardware Resource Summary – STM32F103C8T6

The STM32F103C8T6 is based on a 32-bit ARM Cortex-M3 core operating up to 72 MHz. It provides 64 KB of Flash memory and 20 KB of SRAM. The microcontroller includes three USART peripherals, two SPI interfaces, two I2C interfaces, one advanced-control timer, multiple general-purpose timers, and a 12-bit ADC with multiple input channels. Peripheral functionality is multiplexed across GPIO ports using alternate function mapping, and debugging access is provided through the Serial Wire Debug interface.

Challenges Faced and Insights Gained

Understanding clock configuration highlighted how central the clock tree is to the entire system. GPIO configuration clarified that peripherals cannot function independently of pin configuration. DMA required careful conceptual understanding due to its CPU-independent behavior. I2C emphasized that firmware configuration must respect physical electrical constraints. Together, these learning challenges helped develop a strong mental model of how STM32 hardware blocks interact.

Conclusion

Phase 1 established a solid conceptual foundation in STM32 firmware development using the STM32F103C8T6 microcontroller. Focusing on register-level understanding and configuration flow provided clarity on how peripherals, clocks, GPIOs, and interrupts interact. This phase prepared the groundwork for practical firmware implementation and debugging in subsequent phases.